

IBVAP — Intelligent Border Video Analytics Platform

Complete build guide, SIH 2026 · PS 26187 · Ministry of Home Affairs / SSB

Part 0 — What you are actually building

A **software layer** that sits on top of CCTV cameras that already exist. Video comes in over the network, an AI model looks at every frame, rules decide what counts as an incident, and alerts appear on a dashboard with a saved clip as evidence.

The single most important framing for your pitch:

Smart cameras with built-in analytics cost ₹40,000–₹1,50,000 each. A Border Out Post may already have 8 ordinary cameras worth ₹3,000 each. Replacing them across thousands of posts is financially impossible. Our software gives those existing cameras the same capability for the price of one edge computer per post.

Everything else in the project supports that one sentence.

The eight required capabilities, mapped to what actually builds them

Requirement from the PS	How you build it	Difficulty
Human detection and tracking	YOLO + ByteTrack, one function call	Easy
Vehicle detection and classification	Same call — COCO classes car/truck/bus/motorcycle	Easy
Virtual fence intrusion detection	Your own geometry code on tracked centre points	Easy, ~40 lines
Suspicious activity detection	Dwell time, loitering, wrong-direction, from track history	Medium
Night-time movement detection	Same detection + CLAHE contrast enhancement + time filter	Easy
Real-time alert generation and event logging	FastAPI + SQLite + WebSocket	Medium
Face detection	InsightFace (SCRFD detector + ArcFace embeddings)	Medium

Requirement from the PS	How you build it	Difficulty
ANPR	Plate detector + OCR, needs Indian-plate fine-tuning	Hard — do last

Six of eight come almost free from one model. Treat face and ANPR as stretch goals.

Part 1 — What to learn first

Do these in order. Do not skip ahead; each one is a prerequisite for the next.

1.1 Python fundamentals (skip if comfortable)

Functions, dictionaries, list comprehensions, classes, `with` blocks, virtual environments.

Automate the Boring Stuff — <https://automatetheboringstuff.com/> (free, Chapters 1–6 are enough)

1.2 OpenCV basics

Reading video, showing frames, drawing rectangles and lines, colour spaces, resizing.

Official tutorials — https://docs.opencv.org/4.x/d9/df8/tutorial_root.html

PyImageSearch OpenCV start guide — <https://pyimagesearch.com/start-here/>

The only functions you truly need: `VideoCapture`, `read`, `imshow`, `rectangle`, `putText`, `line`, `resize`, `waitKey`, `VideoWriter`.

1.3 Object detection concepts

You need the *ideas*, not the maths: bounding box, confidence score, class, IoU, non-maximum suppression, mAP.

Ultralytics docs, Predict mode — <https://docs.ultralytics.com/modes/predict/>

Ultralytics Track mode — <https://docs.ultralytics.com/modes/track/>

1.4 Multi-object tracking

Understand *why* detection alone is not enough: a detector tells you “there is a person here” in each frame independently, with no memory. Tracking assigns a persistent ID so you can say “person #7 moved from A to B”, which is what every rule in this project depends on.

1.5 FastAPI

Routes, path and query parameters, Pydantic models, `StreamingResponse`, WebSockets.

Official tutorial — <https://fastapi.tiangolo.com/tutorial/> (the best documentation in Python, work through it directly)

Part 2 — Environment setup

```

bash
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate

pip install ultralytics opencv-python fastapi "uvicorn[standard]" \
    sqlalchemy python-multipart numpy shapely
```

`shapely` handles point-in-polygon tests for your virtual fence — do not write that geometry by hand.

Model choice. As of 2026 the latest Ultralytics release is **YOLO26** (January 2026), which removed non-maximum suppression from the pipeline for genuinely end-to-end inference. **YOLO11** remains the mature, heavily-documented alternative.

For a hackathon, use `yolo11n.pt` or `yolo26n.pt` — the nano variants. They run on a CPU laptop at usable speed. Larger variants (`s`, `m`, `l`, `x`) are more accurate but you do not have the GPU budget to demo them.

```

python
from ultralytics import YOLO
model = YOLO("yolo11n.pt") # downloads once, then cached
```

Do this while you still have internet. After the first run, find the cached `.pt` file and copy it into your project folder, then load it by path. Also run `pip download -r requirements.txt -d ./packages` so you can install offline at the venue.

Part 3 — Build phases

Phase 1 — Detection on screen

Goal: boxes drawn on a video. Nothing else.

```

python
```

```

import cv2
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
cap = cv2.VideoCapture("sample.mp4")

while True:
    ok, frame = cap.read()
    if not ok:
        break
    results = model(frame, verbose=False)[0]
    for box in results boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        label = model.names[int(box.cls)]
        conf = float(box.conf)
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(frame, f"{label} {conf:.2f}", (x1, y1 - 8),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
    cv2.imshow("IBVAP", frame)
    if cv2.waitKey(1) == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()

```

If this runs, your hardest environment problem is already solved.

Phase 2 — Tracking

Add `persist=True` and you get stable IDs. This is the foundation of every rule.

```

python
results = model.track(frame, persist=True, tracker="bytetrack.yaml", verbose=False)[0]

for box in results boxes:
    if box.id is None:
        continue
    track_id = int(box.id)
    cls_name = model.names[int(box.cls)]
    x1, y1, x2, y2 = map(int, box.xyxy[0])
    foot = ((x1 + x2) // 2, y2) # feet, not centre of box

```

Use the bottom-centre point, not the box centre. A person's feet touch the ground plane; the box centre floats at chest height and crosses a drawn line at the wrong moment. This one detail is the difference between an intrusion detector that works and one that fires early.

Phase 3 — The virtual fence

This is the heart of the project and the part that is entirely your own code.

Concept. A fence is a line segment. For any point, compute the sign of the cross product of the line vector and the vector to the point. That sign tells you which side of the line the point is on. Store each track ID's previous side. When the sign flips, someone crossed — and the direction of the flip tells you whether they entered or exited.

```
python
def side_of_line(p, a, b):
    """Which side of line a->b does point p fall on? Returns +1, -1 or 0."""
    cross = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])
    return (cross > 0) - (cross < 0)

class FenceMonitor:
    def __init__(self, a, b):
        self.a, self.b = a, b
        self.last_side = {}

    def update(self, track_id, point):
        now = side_of_line(point, self.a, self.b)
        if now == 0:
            return None
        before = self.last_side.get(track_id)
        self.last_side[track_id] = now
        if before is not None and before != now:
            return "entry" if now > 0 else "exit"
        return None
```

For a **zone** rather than a line — a restricted area, a loitering region — use a polygon instead:

```
python
from shapely.geometry import Point, Polygon

zone = Polygon([(100, 300), (600, 300), (600, 700), (100, 700)])
inside = zone.contains(Point(foot))
```

Loitering follows immediately: record the timestamp when a track ID first enters the zone, and raise an alert when `now - entry_time > 60` seconds and the ID is still inside.

Debouncing matters. Without it, a person standing on the line generates fifty alerts a second. Suppress repeat alerts for the same track ID and rule for at least 10 seconds. Judges notice a flooding alert feed

instantly.

Phase 4 — Alerts, database, evidence clips

Schema — three tables is plenty:

```
❏
sql
CREATE TABLE cameras (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  url TEXT NOT NULL,
  location TEXT,
  is_active BOOLEAN DEFAULT 1
);

CREATE TABLE rules (
  id INTEGER PRIMARY KEY,
  camera_id INTEGER REFERENCES cameras(id),
  rule_type TEXT,          -- 'line', 'zone', 'loiter'
  geometry TEXT,          -- JSON coordinates
  params TEXT             -- JSON, e.g. {"dwell_seconds": 60}
);

CREATE TABLE alerts (
  id INTEGER PRIMARY KEY,
  camera_id INTEGER REFERENCES cameras(id),
  alert_type TEXT,
  object_class TEXT,
  track_id INTEGER,
  confidence REAL,
  timestamp TEXT NOT NULL,
  snapshot_path TEXT,
  clip_path TEXT,
  prev_hash TEXT,
  hash TEXT
);
```

Why SQLite. Single file, no server process, no credentials, ships with Python, survives a laptop reboot. At your scale it is not a compromise. When a judge asks about scale, the honest answer is architecturally correct: *SQLite on the edge device at each BOP, replicating alert metadata to PostgreSQL at the sector headquarters*. That mirrors how real distributed surveillance systems are built.

Evidence clips. Keep a `collections.deque(maxlen=150)` of recent frames. When an alert fires, dump the buffer plus the next 50 frames to an MP4 with `cv2.VideoWriter`. You now have 10 seconds of context around every incident, which is exactly what an investigator needs.

The hash chain. Each alert stores the hash of the alert before it:

```
python
import hashlib, json

def chain_hash(payload: dict, prev_hash: str) -> str:
    blob = json.dumps(payload, sort_keys=True) + prev_hash
    return hashlib.sha256(blob.encode()).hexdigest()
```

Verification walks the table recomputing each hash. If any row was edited or deleted, the chain breaks and the endpoint reports exactly which row. Export the latest hash to a file or print it daily and the whole history before that point is sealed.

This is what makes your project fit the **Blockchain & Cybersecurity** theme it was filed under. Most teams will build pure computer vision and ignore the theme tag entirely.

Phase 5 — API layer

```
python

GET /api/cameras          list cameras
POST /api/cameras         register a camera
DELETE /api/cameras/{id}  remove

GET /api/cameras/{id}/rules  fences and zones for this camera
POST /api/cameras/{id}/rules  save a drawn fence
DELETE /api/rules/{id}

GET /api/alerts           ?camera_id= &type= &from= &to= &limit=
GET /api/alerts/{id}      single alert detail
GET /api/alerts/{id}/clip  download evidence MP4
GET /api/alerts/{id}/snapshot  still image
GET /api/integrity/verify  walk the hash chain
GET /api/stats            counts by type, by hour, by camera

GET /stream/{camera_id}   MJPEG live feed with overlays
WS /ws/alerts             push alerts to dashboard in real time
```

The MJPEG trick — this makes your demo look far more impressive than the effort it costs:

```
python
```

```

from fastapi.responses import StreamingResponse

def frames(camera_id: int):
    while True:
        frame = latest_annotated_frame[camera_id]
        ok, buf = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 70])
        yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n"
              + buf.tobytes() + b"\r\n")

@app.get("/stream/{camera_id}")
def stream(camera_id: int):
    return StreamingResponse(
        frames(camera_id),
        media_type="multipart/x-mixed-replace; boundary=frame")

```

The browser then displays it with nothing more than ``. No WebRTC, no HLS, no video library.

Threading. Run one detection thread per camera writing into a shared dict of latest frames. FastAPI reads from that dict. Never run inference inside a request handler — it will block every other request.

Phase 6 — Dashboard

Plain HTML, CSS and JavaScript is completely acceptable and removes an entire category of build failures. Use React only if two teammates already know it well.

What it must contain:

- A grid of live camera tiles, each an `<img>` pointing at `/stream/{id}`
- Alert feed on the right, newest first, updating over the WebSocket
- Click an alert → modal with the evidence clip and full metadata
- A canvas overlay for drawing the fence with the mouse, POSTing coordinates to `/api/rules`
- Filters: camera, alert type, time range
- An "Verify log integrity" button that calls `/api/integrity/verify` and shows a green tick

The fence drawing tool is the highest-value UI feature. It proves your system is configurable per site rather than hardcoded for one demo video, which is the first thing a sceptical judge probes.

Phase 7 — LAN deployment

Cameras: install **IP Webcam** on Android phones. Each gives you `http://192.168.x.x:8080/video`, openable directly by `cv2.VideoCapture`.

Bind the server to `0.0.0.0`, never `127.0.0.1`: `uvicorn main:app --host 0.0.0.0 --port 8000`

Find your LAN IP with `ipconfig` / `ip addr`; teammates open `http://<that-ip>:8000`

Allow the port through Windows Defender Firewall — a blocked port looks exactly like a code bug

Client isolation on institutional Wi-Fi blocks device-to-device traffic entirely. Carry a phone hotspot or a cheap router as fallback and test it a week early

Downscale streams to 640×480 at ~5 FPS. Four 1080p streams will saturate the Wi-Fi and the CPU

Phase 8 — Stretch goals, only after 1–7 are solid

Face detection and recognition — use InsightFace, which bundles SCRFD for detection and ArcFace embeddings for identity:

```
bash
pip install insightface onnxruntime
```

Store embeddings of watchlist individuals, compare new faces by cosine similarity, threshold around 0.5. Be honest in the pitch that face recognition at typical CCTV distances and angles is unreliable — presenting it as a lead-generation aid rather than positive identification shows maturity.

ANPR — two stages: a plate detector (fine-tune YOLO on an Indian plate dataset) then OCR on the crop. Generic OCR models fail badly on Indian plates because of font variety, script mixing, and non-standard formats. This is well documented in the literature (see Part 5). `easyocr` or `paddleocr` are the practical choices.

Low-light — apply CLAHE (`cv2.createCLAHE`) to the luminance channel before inference. It is three lines and visibly improves night detection. Zero-DCE is the deep-learning alternative if you have time.

Part 4 — Research papers

Read the abstract and introduction of each. You do not need to follow the maths — you need to be able to explain the idea and cite it in your presentation.

Object detection — the lineage

Paper	Why it matters	Link
You Only Look Once (Redmon et al., 2016)	The founding paper. Single-pass detection instead of sliding windows. Read this one properly.	https://arxiv.org/abs/1506.02640
Faster R-CNN (Ren et al., 2015)	The two-stage approach YOLO was reacting against. Useful for comparison slides.	https://arxiv.org/abs/1506.01497
SSD (Liu et al., 2016)	The other major single-shot detector.	https://arxiv.org/abs/1512.02325
Focal Loss / RetinaNet (Lin et al., 2017)	Explains the class-imbalance problem in dense detection.	https://arxiv.org/abs/1708.02002
Ultralytics YOLO Evolution: YOLO26, YOLO11, YOLOv8,	Recent survey covering the current generation and its NMS-free design. Cite this for your model-choice justification.	https://arxiv.org/abs/2510.09653

Paper	Why it matters	Link
YOLOv5 (Sapkota & Karkee, 2026)		
Multi-object tracking		
Paper	Why it matters	Link
SORT (Bewley et al., 2016)	Kalman filter + Hungarian algorithm. The simplest tracker worth understanding.	https://arxiv.org/abs/1602.00763
DeepSORT (Wojke et al., 2017)	Adds appearance embeddings so identities survive occlusion.	https://arxiv.org/abs/1703.07402
ByteTrack (Zhang et al., ECCV 2022)	What you are actually running. Its insight: keep the low-confidence detections instead of discarding them, and match them against existing tracks. Recovers partially occluded people. Read this one.	https://arxiv.org/abs/2110.06864
BoT-SORT (Aharon et al., 2022)	The other tracker Ultralytics ships. Adds camera-motion compensation.	https://arxiv.org/abs/2206.14651
OC-SORT (Cao et al., 2023)	Handles non-linear motion and long occlusions better.	https://arxiv.org/abs/2203.14360
Face detection and recognition		
Paper	Why it matters	Link
FaceNet (Schroff et al., 2015)	Introduced the embedding-plus-distance framing that all modern face recognition uses.	https://arxiv.org/abs/1503.03832
RetinaFace (Deng et al., 2019)	Single-stage dense face localisation with landmarks.	https://arxiv.org/abs/1905.00641
ArcFace (Deng et al., CVPR 2019)	The additive angular margin loss behind InsightFace. The standard baseline for face embeddings.	https://arxiv.org/abs/1801.07698
SCRFD (Guo et al., 2021)	Efficient face detector, the one InsightFace uses by default.	https://arxiv.org/abs/2105.04714

ANPR — read these before promising ANPR in your pitch

Paper	Why it matters	Link
Indian Licence Plate Dataset in the documents why models trained on Chinese or Wild (Tanwar et al., 2021)	16,192 images, 21,683 annotated plates. Explicitly Brazilian plates fail on Indian ones — font styles and plate designs vary too much between countries.	https://arxiv.org/abs/2111.06054
End-to-End ANPR for Indian datasets (Nadiminti et al., BARC, 2022)	From Bhabha Atomic Research Centre. Found that fine-tuning a COCO-pretrained YOLO on Indian vehicle images was more sample-efficient than adapting a CCPD-trained model. Directly relevant, and citing Indian government research strengthens your pitch.	https://arxiv.org/abs/2207.06657
CRNN (Shi et al., 2015)	The convolutional-recurrent architecture underlying most OCR engines.	https://arxiv.org/abs/1507.05717
PP-OCR (Du et al., 2020)	The system behind PaddleOCR, a practical choice for plate text.	https://arxiv.org/abs/2009.09941

Anomaly and suspicious activity detection

Paper	Why it matters	Link
Real-world Anomaly Detection in Surveillance Videos (Sultani et al., CVPR 2018)	Introduced UCF-Crime: 1,900 untrimmed surveillance videos, 13 anomaly classes, ~128 hours. Uses multiple-instance learning with only video-level labels, avoiding frame-by-frame annotation. The canonical citation for this problem.	https://arxiv.org/abs/1801.04264
Survey on Deep Learning for Video Anomaly Detection	Broad overview of the field and its datasets — good for your literature-review slide.	https://arxiv.org/abs/2009.14146

Low-light enhancement

Paper	Why it matters	Link
Zero-DCE (Guo et al., CVPR 2020)	Low-light enhancement with no paired training data. Relevant to the night-time requirement.	https://arxiv.org/abs/2001.06826

Part 5 — Datasets

Dataset	Use	Link
COCO	What your pretrained model already knows — 80 classes including person, car, truck, bus, motorcycle	https://cocodataset.org/
UCF-Crime	Real surveillance footage of anomalies; use clips as demo input	https://www.crcv.ucf.edu/projects/real-world/
MOT17 / MOT20	Tracking benchmarks, for evaluating your tracker quantitatively	https://motchallenge.net/
Indian Licence Plate Dataset	ANPR fine-tuning — see the paper above	https://arxiv.org/abs/2111.06054
Roboflow Universe	Search for pre-annotated Indian number plate sets	https://universe.roboflow.com/
WIDER FACE	Face detection benchmark	http://shuoyang1213.me/WIDERFACE/

For demo footage: search YouTube for CCTV compilations, perimeter security demos, and border patrol footage, and download with `yt-dlp`. Collect 5–6 clips well in advance — the venue Wi-Fi will not cooperate.

Part 6 — Documentation you will reference constantly

Ultralytics docs — <https://docs.ultralytics.com/>

Ultralytics GitHub — <https://github.com/ultralytics/ultralytics>

FastAPI — <https://fastapi.tiangolo.com/>

OpenCV Python — https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

SQLAlchemy ORM tutorial — <https://docs.sqlalchemy.org/en/20/orm/quickstart.html>

Shapely — <https://shapely.readthedocs.io/>

InsightFace — <https://github.com/deepinsight/insightface>

EasyOCR — <https://github.com/JaidedAI/EasyOCR>

Python `hashlib` — <https://docs.python.org/3/library/hashlib.html>

Part 7 — Team split (6 people)

Role	Owns
CV lead 1	Detection, tracking, model integration, night-mode
CV lead 2	Rules engine — fences, zones, loitering, debouncing
Backend	FastAPI, SQLite, hash chain, clip recording, threading
Frontend 1	Dashboard layout, live tiles, alert feed, WebSocket
Frontend 2	Fence drawing canvas, filters, replay modal
Presentation + research	Slides, papers, cost analysis, demo script, deployment story

That last role is not a consolation prize. In Round 1 the presentation *is* the submission, and in the final round the pitch decides as much as the code.

Part 8 — What judges will ask, and your answers

"Why not just buy cameras with built-in analytics?" Cost per post, multiplied by the number of posts along a 15,000 km border. Also vendor lock-in — proprietary systems cannot be integrated with existing command-and-control software. Have a specific rupee figure ready.

"How does this work without internet at a remote BOP?" Everything runs on a local edge device — a Jetson Orin Nano or a small GPU mini-PC. Only alert metadata and short clips travel upstream, not continuous video. Demonstrate this by running your whole system on an isolated hotspot with airplane mode on your laptop.

"How many cameras can one device handle?" Give a measured number, not a guess. Benchmark it: run 1, 2, 4 streams and record the FPS. A real number from your own testing is far more convincing than a claim.

"What about false alarms?" Confidence thresholds, minimum object size, debounce windows, and configurable per-camera rules. Show that you thought about the operator's experience, not just the model's accuracy.

"Why is this filed under Blockchain & Cybersecurity?" This is your opening. Explain the tamper-evident hash chain, the evidentiary value in a court or court-martial, and add RBAC, TLS, and audit logging.

"Is face recognition accurate enough?" Say honestly that at CCTV distances and angles it is not reliable enough for positive identification, and that you present it as an investigative lead requiring human confirmation. Overclaiming here is the fastest way to lose credibility with a Home Ministry panel.

Part 9 — Realistic timeline

Before the internal college round: Phases 1–5 working, plus slides. Record a screen-capture demo as backup in case live demo fails.

If selected for the national round: Phases 6–7 polished, multi-camera on LAN, then Phase 8 stretch goals only if everything else is stable.

The single biggest mistake teams make is attempting all eight capabilities at 40% quality. A complete, reliable pipeline covering intrusion, vehicles, and tamper-proof logging beats a half-broken system that gestures at everything. Put the rest under "future scope" and say so confidently.